TP n°1

Création d'un Moteur RAG Local

Retrieval-Augmented Generation avec *Faiss*, *Gemma 3* et *Ollama*

Objectifs Pédagogiques

Durée : 3 heures
Contexte : Les LLM ont une connaissance figée au moment de leur entrainement. Leur utilisation sur des données postérieures ou privées sont donc limitées par construction.
But : Dans ce TP, vous allez apprendre à créer un système capable de répondre à des questions sur une documentation privée en utilisant la recherche vectorielle et **Gemma 3**.

Phase 1 : Installation et architecture locale

Pour ce TP, nous n'utiliserons pas le Cloud afin à la fois de garantir la confidentialité de vos données, et d'apprendre et de maîtriser l'environnement d'exécution. L'ensemble des opérations seront exécutées sur votre machine. Cela implique de lancer deux composants : le **Serveur** (Ollama) qui sera responsable du chargement des modèles neuronaux et de leur interrogation, et d'un **Client** (un script Python que vous créerez) chargé de la logique d'indexation, de recherche et d'interrogation.

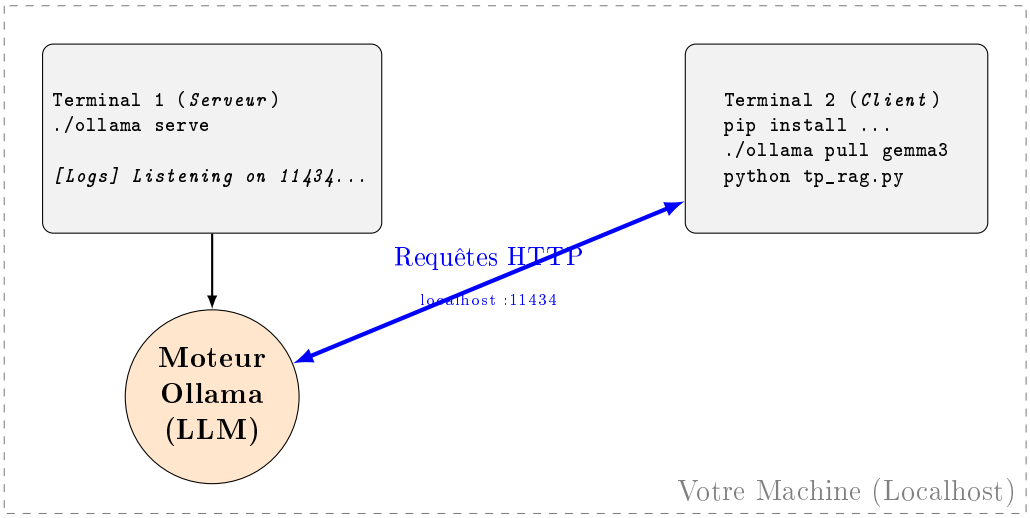

Figure 1 – Architecture d'exécution du TP : un client Python interroge le serveur Ollama localement

Étape 1 : Configuration de l'environnement d'exécution du llm

Nous devons en premier lieu installer l'environnement d'exécution (Ollama + Modèles) et les dépendances Python. N'étant pas administrateur sur les postes de TP, nous allons utiliser des installations locales dans votre dossier utilisateur.

L'ensemble des ressources à télécharger (Ollama + Modèles) sont disponibles sur la page https://scolarite.polytech.univ-nantes.fr/cours/llm/.

Remarque importante : *Un modèle de type Gemma 3 pèse environ 3 Go en version quantisée. Pour économiser la bande passante, vous ne téléchargerez les modèles qu'une seule fois (depuis un serveur local et non depuis Internet). Vous devrez ensuite les importer localement sur votre machine.*
Ne tentez PAS de télécharger les modèles directement depuis Internet *depuis votre poste local, cela reviendrait à saturer le réseau de la salle TP.*

ÉTAPE 1.1 : Installation d'*Ollama* Dans un premiers temps, vous devez installer Ollama. La procédure diffère selon votre système d'exploitation.

Option 1.1 : Si vous travaillez sous Linux

1. **Téléchargement :** Créez un dossier `tp-ollama` dans votre dossier utilisateur et téléchargez Ollama.

```
mkdir ~/tp-ollama && cd ~/tp-ollama
curl -L https://scolarite.polytech.univ-nantes.fr/cours/llm/files/ollama-
    linux-amd64.tgz -o ollama.tgz
tar -xzf ollama.tgz
```

2. **Lancement du Serveur (Terminal 1) :**

```
./bin/ollama serve
# Laissez ce terminal ouvert !
```

3. **Installation Python (Terminal 2) :**

```
cd ~/tp-ollama
python3 -m venv .venv
source .venv/bin/activate
pip install ollama faiss-cpu numpy
```

Option 1.2 : Si vous travaillez sous Windows

L'installateur Windows fonctionne sans droits admin s'il est bien manipulé.

1. Téléchargez `OllamaSetup.exe` (https://scolarite.polytech.univ-nantes.fr/cours/llm/files/OllamaSetup.exe) et lancez-le.
2. **Important :** Si une fenêtre demande un mot de passe Admin, cliquez sur **NON** ou fermez-la. L'installation a souvent déjà réussi dans votre dossier utilisateur.
3. Par défaut, *Ollama* s'installe dans `$env:LOCALAPPDATA\Programs\Ollama`. Les modèles seront stockés dans `$env:USERPROFILE\.ollama\models`. Afin de ne pas surcharger votre profil synchronisé sure le réseau, vous devez modifier le dossier servant au stockage des modèles. Pour celà, ouvrez les paramètres (*Settings...*) de *Ollama* via son icône dans la barre des tâche et changez le paramètre **Model location** vers un dossier dans votre partage *Données* `Z:/`. Par exemple : `Z:/tp-ollama/models`.
4. Si le serveur n'est pas lancé, vous devez utiliser un terminal particulier à ce but.

```
cd $env:LOCALAPPDATA\Programs\Ollama
.\ollama.exe serve
```

Dans la suite de ce sujet de TP, remplacez les commandes Linux par leurs équivalents Windows si nécessaire :

```
# Linux
./bin/ollama <command>

# Windows
$env:LOCALAPPDATA\Programs\Ollama\ollama.exe <command>
```

Étape 1.2 : Installation des modèles manuellement Pour économiser la bande passante, les modèles sont disponibles sur un serveur local à l'école. Vous devez les télécharger une seule fois depuis ce serveur, puis les importer localement.

Dans ce TP, nous utiliserons les modèles suivants :

- **Gemma 3 1B** (`gemma-3-1b-it.gguf`) : Modèle de chat léger (1 milliard de paramètres).
- **Gemma 3 4B** (`gemma-3-4b-it.gguf`) : Modèle de chat plus puissant (4 milliards de paramètres). Ce modèle nécessite un adaptateur de quantification (`mmproj-model-f16.gguf`) pour pouvoir traiter des entrées multimodales (images + texte).
- **Nomic-Embed-Text** (`nomic-embed-text.gguf`) : Modèle d'embedding textuel (768 dimensions) utilisé pour la vectorisation.

Téléchargement depuis le serveur local (Terminal 2) : Exécutez les commandes suivantes dans le **Terminal 2** pour télécharger les fichiers `.gguf` depuis le serveur local (ou téléchargez-les manuellement depuis le site ressource des TP) :

```
mkdir -p ~/tp-ollama/models
cd ~/tp-ollama/models
curl -O https://scolarite.polytech.univ-nantes.fr/cours/llm/files/gemma-3-1b-it.
    gguf
curl -O https://scolarite.polytech.univ-nantes.fr/cours/llm/files/gemma-3-4b-it.
    gguf
curl -O https://scolarite.polytech.univ-nantes.fr/cours/llm/files/nomic-embed-
    text.gguf
```

Importation locale des modèles Une fois les fichiers `.gguf` téléchargés, vous devez les importer dans Ollama. Pour celà, vous devez créer un fichier *Modelfile* en fonction des modèles que vous souhaitez installer.

- ***Modelfile* pour Gemma 3 1B** (fichier `Modelfile-gemma3-1b`) :

  ```
  FROM ./gemma-3-1b-it.gguf
  ```

- ***Modelfile* pour Gemma 3 4B** (fichier `Modelfile-gemma3-4b`) :

  ```
  FROM ./gemma-3-4b-it.gguf
  ADAPTER ./mmproj-model-f16.gguf
  ```

- ***Modelfile* pour Nomic-Embed-Text** (fichier `Modelfile-nomic-embed-text`) :

  ```
  FROM ./nomic-embed-text.gguf
  ```

Exécutez les commandes suivantes dans le **Terminal 2** :

```
./bin/ollama create gemma3:4b -f Modelfile-gemma3-4b
./bin/ollama create gemma3:1b -f Modelfile-gemma3-1b
./bin/ollama create nomic-embed-text -f Modelfile-nomic-embed-text
```

L'importation peut prendre plusieurs minutes. Une fois terminée, vous pouvez vérifier la liste des modèles installés avec :

```
./bin/ollama list
```

Étape 2 : Configuration du contexte d'exécution Python

Étape 2.1 : Installation de l'environnement Python Commencez par créer un environnement virtuel Python dans le dossier `tp-ollama` (si ce n'est pas déjà fait) et installez les dépendances nécessaires :

```
cd ~/tp-ollama

python3 -m venv .venv
source .venv/bin/activate
# ou sous Windows : .\.venv\Scripts\Activate

pip install ollama faiss-cpu numpy
```

Étape 2.2 : Vérification de l'installation Pour vérifier que tout fonctionne, dans le **Terminal 2**, lancez le script de test présent sur madoc

Vous devriez obtenir une réponse cohérente de Gemma 3. Si ce n'est pas le cas, vérifiez que le serveur *Ollama* (Terminal 1) est bien en cours d'exécution.

Phase 2 : Vectorisation d'une base documentaire

Afin de créer un moteur RAG (Retrieval-Augmented Generation), nous devons d'abord disposer d'une base de connaissances privée. Le modèle de llm n'ayant pas été entraîné sur cette base, il ne la connaît pas nativement.

Nous allons donc devoir indexer cette base documentaire afin de la rendre accessible au modèle via des requêtes. L'idée est de transformer chaque document en un vecteur numérique (embedding) représentant de manirère condensée la sémantique du texte. Cette transformation est réalisée à l'aide d'un modèle d'embedding spécialisé qui produit des vecteurs dans un espace de haute dimension (ici 768 dimensions) où la proximité géométrique entre deux vecteurs reflète la similarité sémantique entre les textes correspondants.

Étape 3 : Préparation de la base documentaire

Dans un premier temps, nous allons simuler une documentation pour une machine imaginaire : la *Turbo-Encabulator*. Reprenez le fichier `tp_rag.py` présent sur *madoc* ou recopiez le source 1.

Ce fichier contient le squelette de notre moteur RAG. Nous allons commencer par la vectorisation de notre base de documentation (fictive dans un premier temps).

Vous pouvez vérifier la dimension des vecteurs produits par le modèle d'embedding. Normalement, elle doit être de 768 pour le modèle *nomic*.

```
print(f"Dimension vecteur : {len(get_embedding('Test'))}")
```

Question 1 : Création des embeddings

Créez une liste `embeddings` contenant les embeddings de chaque document de la base `knowledge_base`. Remplissez cette liste dans une fonction `generate_embeddings` en utilisant la fonction `get_embedding`.

Les emeddings doivent être stockés sous forme de tableau numpy pour faciliter les calculs ultérieurs. En effet, les opérations vectorielles sont optimisées dans numpy et permettent de manipuler efficacement des matrices de grande dimension.

Pour celà, il suffit d'écrire : `embeddings_np = np.array(embeddings)`.

Étape 4 : Indexation naïve par similarité cosinus

Pour calculer la similarité entre deux vecteurs, nous utiliserons la similarité *cosinus*, définie comme le cosinus de l'angle entre deux vecteurs dans un espace vectoriel. La similarité cosinus est une mesure de similarité qui évalue à quel point deux vecteurs pointent dans la même direction, indépendamment de leur magnitude. Elle est particulièrement utile dans le traitement du langage naturel et la recherche d'information, car elle permet de comparer des documents ou des phrases

Listing 1 Structure du fichier `tp_rag.py`

```
import ollama
import numpy as np

def get_embedding(text):
  # On utilise un modèle d'embedding dédié (nomic), pas le modele de chat
  response = ollama.embeddings(model='nomic-embed-text', prompt=text)
  return response['embedding']


if __name__ == "__main__":
  # Notre base de documentation (que Gemma ne connait pas nativement)
  knowledge_base = [
    "La Turbo-Encabulator 3000 utilise un stator logarithmique pré-activé.",
    "Pour redémarrer le système d'urgence, maintenez le bouton rouge pendant 5
   secondes, puis tournez la clé bleue.",
    "L'erreur 404 sur cette machine indique une surchauffe du condensateur
   fluxionnel principal.",
    "La maintenance des roulements à billes doit être effectuée tous les 150
   cycles par un technicien certifié de niveau 2.",
    "Le voltage nominal d'entrée est de 220V, mais l'appareil tolère des
   fluctuations entre 210V et 240V.",
    "En cas de fuite de liquide réfrigérant (couleur verte), évacuez la zone imm
   édiatement.",
    "Le module Wi-Fi se connecte uniquement sur la bande 2.4GHz avec le
   protocole WPA2.",
    "L'interface tactile peut se bloquer si l'opérateur porte des gants en latex
     non conducteurs."
  ]
```

en se concentrant sur leur contenu sémantique plutôt que sur leur taille ou leur longueur. La formule de la similarité cosinus entre deux vecteurs A et B est donnée par :

$$\text{similarité_cosinus}(A, B) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Question 2 : Similarité *cosinus*

Créez une fonction `cosine_similarity` qui prend en entrée deux vecteurs numpy et retourne leur similarité cosinus.

Utilisez la fonction `np.dot` pour le produit scalaire et `np.linalg.norm` pour calculer la norme (longueur) des vecteurs.

Question 3 : Fonction de recherche naïve

Créez une fonction `naive_search` qui prend en entrée une question textuelle, calcule son embedding, puis compare cet embedding avec ceux de la base documentaire en utilisant la similarité cosinus. La fonction doit retourner le document le plus similaire.

Étape 5 : Indexation Industrielle avec *Faiss*

Pour passer à l'échelle (millions de documents), une boucle Python est trop lente. Pour améliorer les performences, il est nécessaire d'utiliser une bibliothèque spécialisée dans la recherche de voisins les plus proches (KNN - K Nearest Neighbors) dans des espaces de haute dimension.

Dans la suite de ce TP, nous utiliserons la bibliothèque *Faiss* (Facebook AI Similarity Search). Cette bibliothèque open-source est développée par Facebook AI Research et est conçue pour effectuer des recherches efficaces de similarité et de clustering sur de grands ensembles de données

vectorielles. Elle est particulièrement adaptée pour les applications de recherche d'information, de recommandation et de traitement du langage naturel où les données sont souvent représentées sous forme de vecteurs dans des espaces de haute dimension.

Faiss propose plusieurs types d'index. Nous utiliserons ici un index "plat" (non compressé) pour la simplicité et la distance Euclidienne (L2) : `IndexFlatL2`.

En NLP (Natural Language Processing), on préfère généralement le Cosinus, mais *Faiss* utilise par défaut la Distance Euclidienne (L2). Cependant, il existe une relation mathématique entre les deux mesures qui permet d'obtenir des résultats équivalents sous certaines conditions (voir figure 2 et encadré de démonstration ci-dessous).

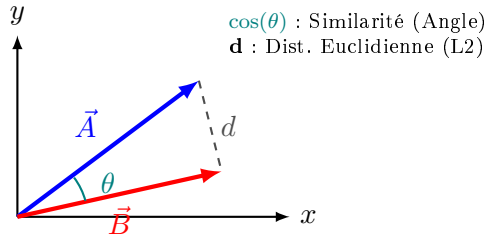

FIGURE 2 – Comparaison géométrique : Angle vs Distance

Démonstration Ingénieur : L'équivalence L2 / Cosinus

Le classement (ranking) produit par la distance Euclidienne est **identique** à celui du Cosinus, **à condition que les vecteurs soient normalisés** (longueur = 1).
Preuve : Soient deux vecteurs normalisés $\vec{A}$ et $\vec{B}$ ($\|\vec{A}\| = \|\vec{B}\| = 1$). La distance Euclidienne au carré est :

$$\|\vec{A} - \vec{B}\|^2 = (\vec{A} - \vec{B}) \cdot (\vec{A} - \vec{B})$$

$$\|\vec{A} - \vec{B}\|^2 = \|\vec{A}\|^2 + \|\vec{B}\|^2 - 2(\vec{A} \cdot \vec{B})$$

Comme les vecteurs sont de norme 1 :

$$\|\vec{A} - \vec{B}\|^2 = 1 + 1 - 2(\vec{A} \cdot \vec{B})$$

$$\|\vec{A} - \vec{B}\|^2 = 2 - 2\cos(\theta)$$

Conclusion : La distance d dépend uniquement de $\cos(\theta)$. Comme la fonction $f(x) = 2 - 2x$ est strictement décroissante :

Maximiser le Cosinus $\Leftrightarrow$ Minimiser la Distance L2

Faiss utilise L2 car les processeurs modernes calculent les distances plus vite que les produits scalaires normalisés.

Faiss exige des données en `float32` pour des raisons de performance. De plus, pour garantir l'équivalence entre la distance L2 et la similarité cosinus, il est impératif de normaliser les vecteurs avant l'indexation et la recherche.

QUESTION 4 : Construction de l'index *Faiss*

Créez une fonction `faiss_index` qui construit un index *Faiss* à partir des embeddings numpy. Assurez-vous de convertir les embeddings en `float32` et de les normaliser avant de les ajouter à l'index.

Utilisez le code d'exemple fourni dans le listing 2 pour effectuer cette intégration.

Listing 2 Intégration de *Faiss* pour l'indexation

```
import faiss

# 1. Conversion indispensable pour \textit{Faiss}
embeddings_f32 = embeddings_np.astype('float32')

# 2. Normalisation des vecteurs (Etape CRUCIALE pour l'equivalence)
# Faiss possede une fonction optimisee pour ca
faiss.normalize_L2(embeddings_f32)

# 3. Creation de l'index
dimension = embeddings_f32.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(embeddings_f32)
print(f"Index Faiss construit avec {index.ntotal} documents.")
```

Question 5 : Recherche avec *Faiss*

Créez une fonction `faiss_search` qui prend en entrée une question textuelle, calcule son embedding, normalise cet embedding, puis utilise l'index *Faiss* pour retrouver les documents les plus similaires. La fonction doit retourner les documents correspondants aux indices renvoyés par *Faiss*.

Utilisez le code d'exemple fourni dans le listing 3 pour effectuer cette intégration.

Listing 3 Intégration de *Faiss* pour la recherche

```
# 4. Recherche
# Normalisation de la requete 'query' (non faite ici)

# Recherche du top-k (k = 2  : Nombre de voisins a recuperer)
k=2
distances, indices = index.search(query_f32, k)

print("\n--- Recherche Faiss ---")
retrieved_docs = []
for i in range(k):
    idx = indices[0][i]
    dist = distances[0][i]
    doc = knowledge_base[idx]
    retrieved_context.append((dist, doc))

# Note : Ici 'distance' est un score L2 (plus petit = mieux)
```

Phase 3 : Génération Augmentée avec Gemma 3

Nous avons maintenant un moteur de recherche vectorielle fonctionnel. Nous allons l'utiliser pour créer un système RAG complet en interrogeant *Gemma 3* avec le contexte retrouvé. L'utilisation d'un LLM permet de formuler des réponses plus naturelles et contextuelles que de simples extraits de texte.

Étape 6 : Interrogation de Gemma 3 avec le contexte

Nous allons maintenant interroger *Gemma 3* en lui fournissant le contexte des documents retrouvés par notre moteur de recherche. Pour celà, nous allons construire un prompt qui inclut les documents pertinents avant de poser la question selon le modèle fourni dans le listing 4.

Dans ce prompt, nous demandons explicitement à *Gemma 3* d'utiliser uniquement le contexte fourni pour formuler sa réponse. Si l'information n'est pas présente dans le contexte, le modèle doit répondre "Je ne sais pas". Cette consigne est cruciale pour éviter que le modèle n'invente des réponses basées sur ses connaissances préalables. Ces instructions aident à maintenir la fiabilité et la pertinence des réponses générées. Les tirets avant chaque document aident à structurer le contexte et à le rendre plus lisible pour le modèle.

Il est important de comprendre qu'il existe des formulations de prompt plus ou moins efficaces. La formulation présentée ici est une approche simple et directe, mais d'autres techniques de prompt engineering peuvent être explorées pour améliorer les performances du modèle dans des scénarios spécifiques.

Listing 4 Construction du prompt RAG pour Gemma 3

```
context_str = "\n".join([f"- {d}" for d in retrieved_docs])

system_prompt = f"""
Tu es un expert technique. Utilise UNIQUEMENT le contexte suivant pour repondre.
Si la reponse n'est pas dans le contexte, dis "Je ne sais pas".

CONTEXTE :
{context_str}

QUESTION :
{query}
"""

response = ollama.chat(model='gemma3:4b', messages=[
    {'role': 'user', 'content': system_prompt}
  ])
print(f"\n[Reponse Gemma 3] : {response['message']['content']}")
```

Question 6 : Intégration complète du RAG

Ajouter la fonction `rag_query` qui prend en entrée une question utilisateur, utilise *Faiss* pour retrouver les documents pertinents, construit le prompt RAG, interroge *Gemma 3* et retourne la réponse finale.

Question 7 : Personnalisation de la requête

Ajouter une interface utilisateur simple (input) pour poser des questions à votre moteur RAG.

Phase 4 : *Bonus* - RAG Multimodal avec Gemma 3

Le défi maintenant est d'utiliser notre moteur de recherche textuel (*Faiss*) pour retrouver des images (schémas, graphiques).

Gemma 3 est un modèle multimodal. Celà signifie qu'il est techniquement capable de traiter plusieurs types de données, comme du texte et des images. Cependant, dans le contexte d'un moteur RAG, l'indexation et la recherche sont souvent limitées au texte. En effet, les modèles d'embedding et les moteurs de recherche vectorielle comme *Faiss* sont généralement conçus pour fonctionner avec des données textuelles et ne supportent pas directement les images.

Il existe cependant des stratégies pour contourner cette limitation et intégrer des images dans un pipeline RAG. L'idée principale est de convertir les images en descriptions textuelles avant de les indexer.

Étape 7 : Le Concept : Le "Pont Sémantique"

Nos outils (*Faiss*, Nomic-Embed) ne comprennent que le texte. Pour indexer une image, nous devons la traduire en texte. Nous utiliserons pour celà notre modèle multimodal afin de décrire l'image en mots. Ce texte sera ensuite vectorisé et indexé comme n'importe quel autre document textuel.

Stratégie "Captioning-based Retrieval" :

1. **Ingestion :** Gemma 3 observe l'image et génère une description détaillée.
2. **Indexation :** Nous vectorisons cette description (et non l'image elle-même).
3. **Recherche :** L'utilisateur pose une question textuelle, on retrouve la description la plus proche.
4. **Génération :** On envoie l'image originale (retrouvée via son chemin) à Gemma 3 pour l'analyse finale.

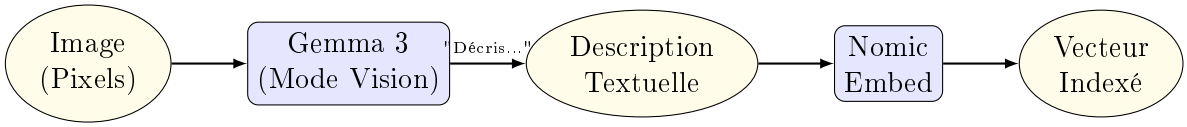

Figure 3 – Pipeline d'Indexation Multimodale

Question 8 : Description multimodale

Créez une fonction `generate_description_for_image` qui utilise Gemma 3 pour générer une description textuelle d'une image donnée. Utilisez le code du listing 5 comme référence.

Remarque : Assurez-vous que le modèle Gemma 3 4B est bien installé avec l'adaptateur multimodal. Le texte de description doit être concis (200 mots maximum) pour éviter de dépasser les limites de tokens lors de l'indexation.

Listing 5 Utilisation de Gemma 3 pour décrire une image

```
# Utiliser Gemma 3 pour décrire l'image
response = ollama.chat(model='gemma3:4b', messages=[{
    'role': 'user',
    'content': 'Décris cette image en détails et en 200 mots maximum.',
    'images': [path]
  }])
description = response['message']['content']
embedding = get_embedding(description)
```

Question 9 : Implémentation du pipeline multimodal

Créez une nouvelle fonction `generate_multimodal_embeddings` afin de demander à Gemma 3 de générer une description textuelle lorsque le document est une image (son url), avant de vectoriser cette description.

Question 10 : Interrogation finale

Lors de la génération de la réponse, si le document le plus pertinent est une image, incluez cette image dans le prompt envoyé à Gemma 3 pour qu'elle puisse l'analyser.

Pour aller plus loin

Ce pipeline est la base des moteurs de recherche d'entreprise modernes. Pour améliorer la précision, on indexe souvent **à la fois** le texte extrait par OCR (Lecture) et la description visuelle (Compréhension) générée par le LLM.

Question d'Expert : Et les embeddings d'images ?

Pourquoi ne pas récupérer directement le vecteur de l'image ?
Le vecteur "visuel" interne de Gemma 3 n'est pas aligné mathématiquement avec le vecteur "textuel" de Nomic. On ne peut pas calculer de distance entre eux. **Solution :** On utilise le "Captioning" (Image → Texte) pour revenir dans un espace commun.

Phase 5 : Vers la Production (Comparatif)

Dans ce TP, nous avons utilisé ***Faiss*** comme une librairie bas niveau. Dans l'industrie, on utilise des **Bases de Données Vectorielles** (Vector Databases) comme *ChromaDB, Qdrant, Milvus* ou *pgvector*.

Fonctionnalité	***Faiss* (Librairie)**	**Vector DB (Chroma, Qdrant...)**
Stockage	RAM (par défaut)	Disque (Persistant)
Données	Vecteurs uniquement	Vecteurs + Texte + Métadonnées
Filtrage	Impossible	"Donne-moi les docs 'PDF' proches de X"
Architecture	Processus local	Client-Serveur (Multi-utilisateurs)

Table 1 – Comparatif : Moteur d'indexation vs Base de données

Pourquoi avons-nous utilisé *Faiss* ? Pour comprendre la mathématique. Une Vector DB n'est souvent qu'une "coquille" de gestion autour d'un index *Faiss* ou HNSW.